LAPORAN PRAKTIKUM MIKROKONTROLER

KONVERSI DATA

BAHASA ASSEMBLY



Elita Lestari Setianto NIM 201344008

Kelas 3 NK

Tanggal Percobaan : Rabu, 14 September 2022

Tanggal Penyerahan Laporan : Rabu, 21 September 2022

Instruktur : 1. Ferry Satria, BSEE, M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022

A. TUJUAN

1. Untuk memenuhi tugas mata kuliah Mikrokontroler.
2. Mahasiswa dapat mengetahui bahasa pemograman Assembly.
3. Mahasiswa mampu melakukan instalasi dan menggunakan *software* AVR Studio 4 untuk simulasi pemrogaman mikrokontroler.
4. Mahasiswa mampu memahami konsep Bahasa Pemrograman Assembly dan menjalankannya di *software* AVR Studio 4.
5. Mahasiswa mampu memahami konsep dan membuat program BCD *Up down Counter* pada *software* AVR Studio 4 serta disimulasikan pada *software* Proteus.

B. PENDAHULUAN

1. Software AVR Studio

Software CodeVisionAVR merupakan perangkat lunak kompailer C dan merupakan pembangkit program otomatis. Software CodeVisionAVR didesain untuk mikrokontroller keluarga atmel AVR. Software CodeVisionAVR dapat berjalan pada sistem operasi windows 95, 98, Me, NT 4, 2000 dan windows XP. Kompailer C melakukan pendekatan elemen dari bahasa ANSI C sesuai dengan arsitektur dari AVR, dengan beberapa fitur yang ditambahkan. Integrated Development Environment (IDE) mendukung software AVR Chip In-System Programmer sehingga memungkinkan untuk melakukan transfer secara otomatis dari program ke chip mikrokontroller setelah berhasil dikompilasi atau diassembly. Software InSystem Programmer didesain untuk dapat bekerja dan mendukung tipe chip AVR STK500/AVRISP/AVRProg, Kanda Systems STK200+/300, Dontronics DT006, Vogel Elektronik VTEC-ISP, Futurlec JRAVR dan MicroTronics ATCPU/Mega2000 programmers/development boards. Software CodeVisionAVR juga mempunyai fasilitas CodeWizardAVR, yang digunakan untuk membantu menuliskan dengan cepat secara otomatis kedalam program. Dengan CodeWizardAVR membantu kita untuk membuat kerangka program dan menuliskan fungsi-fungsi:

A. Mengatur External memory
B. Mengidentifikasi sumber reset Chip

C. Inisialisai Port Input/Output3]
D. Inisialisasi External Interrupts
E. Inisialisasi Timers/Counters
F. Inisialisasi Watchdog Timer
G. Inisialisasi UART (USART) and interrupt driven buffered serial communication
H. Inisialisasi Analog Comparator
I. Inisialisasi ADC
J. Inisialisasi antarmuka SPI
K. Inisialisasi antarmuka Two Wire
L. Inisialisasi antarmuka CAN
M. Inisialisasi I2C Bus, LM75 Temperature Sensor, DS1621 Thermometer/Thermostat dan PCF8563, PCF8583, DS1302, DS1307 Real Time Clocks
N. Inisialisasi 1 Wire Bus and DS1820, DS18S20 Temperature Sensors
O. Inisialisasi LCD module

2. Konversi Data

Konversi data merupakan pengubah format data komputer dari satu format ke format lainnya yang akan di data lalu dikodekan atau dioperasikan dengan berbagai cara serta dibuat berdasarkan standar tertentu.[1] Ada berbagai cara untuk mengonversi suatu dokumen

1. Desimal ke Biner

Cara mengkonversi bilangan desimal ke biner adalah dengan cara membagi bilangan desimal dengan 2 (basis bilangan biner) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan biner. Nilai konversinya adalah urutan sisa hasil bagi dari yang paling akhir.

Contoh :

$88_{(10)} = \ldots_{(2)}$

· 88 : 2 = 44 sisa 0

· 44 : 2 = 22 sisa 0

· 22 : 2 = 11 sisa 0

· 11 : 2 = 5 sisa 1

- 5 : 2 = 2 sisa 1
- 2 : 2 = 1 sisa 0
- 1 : 2 = 0 sisa 1

Maka hasil Konversinya = 1011000 (ditulis dengan urutan dari bawah ke atas)

2. Desimal ke Octal

Cara mengkonversi bilangan desimal ke Oktal adalah dengan cara membagi bilangan desimal dengan 8 (basis bilangan oktal) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan oktal. Nilai konversinya adalah urutan sisa hasil bagi dari yang paling akhir.

Contoh :

1402(10) = … (8)

- 1402/8 = 175 sisa 2
- 175/8 = 21 sisa 7
- 21/8 = 2 sisa 5
- 2/8 = 0 sisa 2

Maka hasil konversinya = 2572 (ditulis dari bawah)

3. Desimal ke Hexadecimal

Cara mengkonversi bilangan desimal ke hexadesimal adalah dengan cara membagi bilangan desimal dengan 16 (basis bilangan hexadesimal) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan heksadesimal. Apabila sisa bagi > 9 maka angkanya dirubah menjadi huruf. Untuk sisa bagi berjumlah 10 = A, 11 = B, 12 = C, 13 = D, 14 = E, 15 = F.

Contoh :

19889(10) = … (16)

· 19889/16 = 1243 sisa 1

· 1243/16 = 77 sisa 11 (B)

· 77/16 = 4 sisa 13 (D)

· 4/16 = 0 sisa 4

Maka hasil konversinya = 4DB1 (ditulis dari bawah)

TABEL KONVERSI BILANGAN

Decimal (x_{10})	Biner (x_2)	Octal (x_8)	Hexa (x_{16})
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

4. Biner ke Octal

Cara mengkonversi bilangan biner ke oktal yakni dengan mengelompokan bilangan biner menjadi 3 kelompok **dimulai dari bilangan biner yang paling kanan**. Setelah dikelompokan barulah kita dapat mengkonversi menjadi bilangan Oktal.
Contoh :

11001101(2) = … (8) ===> 011 001 101

011 = 3 (lihat tabel konversi di atas)

001 = 1

101 = 5

Hasil Konversi : 315 (ditulis dari atas ke bawah)

5. Biner ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan cara mengalikan satu persatu bilangan dengan 2 (basis bilangan biner) pangkat 0, pangkat 1 dan seterusnya sesuai dengan banyaknya bilangan biner yang akan di konversi dan perhitungannya dimulai dari bilangan biner yang paling kanan.

Contoh :

$00011_{(2)} = \ldots_{(10)}$

$= (1x2^0) + (1x2^1) + (0x2^2) + (0x2^3) + (0x2^4)$

$= 1 + 2 + 0 + 0 + 0$

$= 3$

Hasil Konversi : 3

6. Biner ke Hexadecimal

Cara mengkonversi bilangan biner ke hexadesimal tekniknya hampir sama dengan cara konversi bilangan biner ke oktal. **Yang membedakan ada pada pengelompokan bilangan binernya, pada bilangan oktal dalam satu kelompok terdiri dari 3 bit bilangan biner, sedangkan pada hexadesimal dalam satu kelompok terdiri dari 4 bit bilangan biner**.
Contoh :

$10100_{(2)} = \ldots_{(16)}$

· 0001 = 1 (lihat tabel konversi di atas)

· 0100 = 4

Hasil Konversi = 14 (ditulis dari atas ke bawah)

7. Octal Ke Desimal

Cara mengkonversi bilangan oktal ke desimal adalah dengan cara mengalikan satu persatu bilangan dengan 8 (basis bilangan oktal) dengan pangkat 0, 1 dan seterusnya dimulai dari bilangan oktal yang paling kanan. Kemudian hasil dari semua pengalian dijumlahkan.

Contoh :

$62_{(8)} = ..._{(10)}$

· $2x8^0 = 2$

· $6x8^1 = 48$

2+48 = 50

Hasil Konversi = 50

8. Octal Ke Biner

Cara mengkonversi bilangan oktal ke biner adalah dengan **memecah** terlebih dahulu bilangan oktal kedalam satuan bilangan, kemudian masing-masing bilangan diubah kedalam bentuk 3 bit bilangan biner, dengan cara membagi setiap satuan bilangan tersebut dengan 2 (basis bilangan biner). Jika hasil konversi hanya menghasilkan 2 digit bilangan biner, maka harus ditambahkan 0 di sebelah kirinya, supaya bilangan binernya menjadi 3 digit (bit).
Contoh :

$145_{(8)} = ..._{(2)}$

1 = 001 (lihat tabel konversi)

4 = 100

5 = 101

Hasil Konversi = 001100101 (ditulis dari atas ke bawah)

9. Octal Ke Hexadecimal

Cara mengkonversi bilangan oktal ke hexadesimal terdiri dari dua tahap yaitu:

- Pertama, mengkonversi terlebih dahulu setiap bit bilangan oktal ke bilangan biner

- Kedua, hasil konversi ke bilangan biner kemudian di konversikan ke bilangan hexadesimal

Singkatnya seperti ini: Oktal --> Biner --> Hexadesimal.

Contoh :

$46_{(8)}$ = ... (16)
Diubah ke Biner

4 = 100

6 = 110

Diubah Ke Hexadecimal

0100 = 2

0110 = 6

Hasil Konversi = 26

10. Hexadecimal ke Biner

Sama dengan cara konversi bilanga octal ke biner, **bedanya kalau bilangan octal binernya harus 3 bit, sedangkan bilangan hexadesimal binernya terdiri dari 4 bit**. Misal kita konversi 2 hexa menjadi biner hasilnya bukan 10 melainkan 0010
Contoh :

$9A_{(16)}$ = ... (2)
9 = 1001 (lihat tabel konversi)

A/10 = 1010

Hasil Konversi = 10011010 (ditulis dari atas)

11. Hexadecimal ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan mengalikan satu-satu bilangan dengan 16 (basis hexa) pangkat 0 atau 1 atau 2 dst., dimulai dari bilangan paling kanan. Kemudian hasilnya dijumlahkan.

Contoh :

$7A_{(16)}$ = ... (10)
= $(7x16^1) + (10x16^0)$

= 112 + 10

= 122

Hasil Konversi = 122

12. Hexadecimal ke Oktal

Begitu juga dengan konversi hexa desimal ke octal yakni dengan mengubah bilangan hexa ke bilangan biner, kemudian diubah menjadi bilangan octal. Ringkasnya: hexa->biner->octal

Contoh :

$AF_{(16)} = ..._{(10)}$
Diubah ke Biner

A/10 = 1010 (lihat tabel konversi)

F/15 = 1111 ==> hasil konversinya 10101111

Diubah ke Oktal ==> 010 101 111

010 = 2 (lihat tabel konversi)

101 = 5

111 = 7

Hasil Konversi = 257 (ditulis dari atas)

3. Konfigurasi Pin Mikrokontroler AVR ATMega8535

Mikrokontroler ATMega8535 memiliki 40 pin untuk model PDIP, dan 44 pin untuk model TQFP dan PLCC. Nama-nama pin pada mikrokontroler ini adalah

VCC untuk tegangan **pencatu daya positif.**

- GND untuk tegangan pencatu daya negatif.

- PortA (PA0 - PA7) sebagai port **Input**/Output dan memiliki kemampuan lain yaitu sebagai input untuk **ADC**
- PortB (PB0 – PB7) sebagai port Input/Output dan juga memiliki kemampuan yang lain.
- PortC (PC0 – PC7) sebagai port Input/Output untuk ATMega8535.
- PortD (PD0 – PD7) sebagai port Input/Output dan juga memiliki kemampuan yang lain.
- RESET untuk memperagakan reset program dalam mikrokontroler.
- XTAL1 dan XTAL2 untuk input pembangkit sinyal clock.
- AVCC untuk pin masukan tegangan pencatu daya untuk ADC.
- AREF untuk pin tegangan rujukan ADC.

Pengisian Program pada Mikrokontroler AVR

Untuk memperagakan pemrograman dalam mikrokontroler AVR, Atmel telah menyediakan software khusus yang bisa diunduh dari website resmi Atmel. Software tersebut adalah AVRStudio. Software ini memakai bahasa **assembly** sebagai bahasa perantaranya. Selain AVRStudio, benar beberapa software pihak ketiga yang bisa digunakan untuk membuat program pada AVR. Software dari pihak ketiga ini memakai **bahasa pemrograman tingkat tinggi** seperti bahasa C, **Java**, atau Basic. Untuk memperagakan pemindahan dari komputer ke dalam chip, bisa digunakan beberapa cara seperti memakai kabel JTAG atau memakai STNK buatan Atmel.

Input/Output pada Mikrokontroler AVR ATMega8535

Mikrokontroler memiliki arsitektur RISC 8 bit, di mana semua instruksi dikemas dalam kode 16-bit(16-bits word)dan sebagian luhur instruksi dieksekusi dalam satu siklus clock. Mikrokontroler ATMega 8535 memiliki aliran I/O sebagai berikut:

A. Port A(PA0 – PA7) merupakan pin I/O dua arah dan pin masukkan ADC
B. Port B(PB0 – PB7) merupakan pin I/O dua arah dan pin fungsi khusus, yaitu timer/counter, komparator analog, dan SPI.
C. Port C(PC0 – PC7) merupakan pin I/O dua arah dan pin fungsi khusus, yaitu TWI, komparator analog, dan timer oscillator.
D. Port D(PD0 – PD7) merupakan pin I/O dua arah dan pin fungsi khusus, yaitu komparator analog, interupsi eksternal, komunikasi serial.

4. Counter

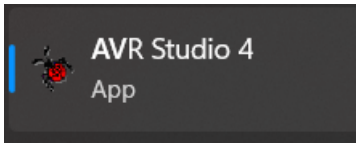
Counter adalah sebuah rangkaian digital yang mengeluarkan urutan state atau biner tertentu, yang merupakan aplikasi dari beberapa pulsa inputnya. Pulsa input tersebut dapat berupa pulsa clock atau pulsa yang dibangkitkan oleh sumber eksternal dan terjadi pada interval waktu tertentu.

Counter banyak diaplikasikan pada peralatan yang berhubungan dengan teknologi digital dan biasanya digunakan untuk aplikasi menghitung jumlah kemunculan sebuah kejadian/event atau untuk menghitung pembangkit pulsa. Aplikasi ini dapat dilihat seperti pada aplikasi sisa parkiran dalam gedung, aplikasi penghitung jumlah barang, aplikasi penghitung jumlah pengunjung perpustakaan dan lain sebagainya. Sebuah n binary counter dapat dibentuk dari n buah flip flop dan dapat menghitung dari 0 sampai 2n-1. Maksudnya adalah apabila kita mendesain counter yang mengeluarkan tiga inputan maka kita harus menggunakan tiga flip – flop, atau sesuai dengan kebutuhan.

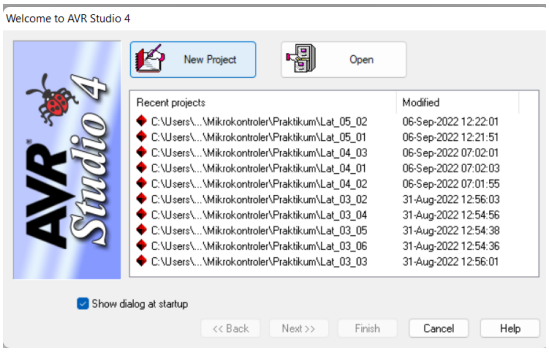
Counter Up adalah serangkaian flip-flop yang di hubungkan secara seri dengan cara output flip-flop yang pertama menjadi input flip-flop berikutnya Pulsa dari clock menjadi iput untuk flip-flop yang pertama dan akan menyebabkan perubahan pada kondisi output untuk saat yang di kehendaki (correct edge) Counter Down berfungsi untuk menghitung secara maju

C. LANGKAH KERJA

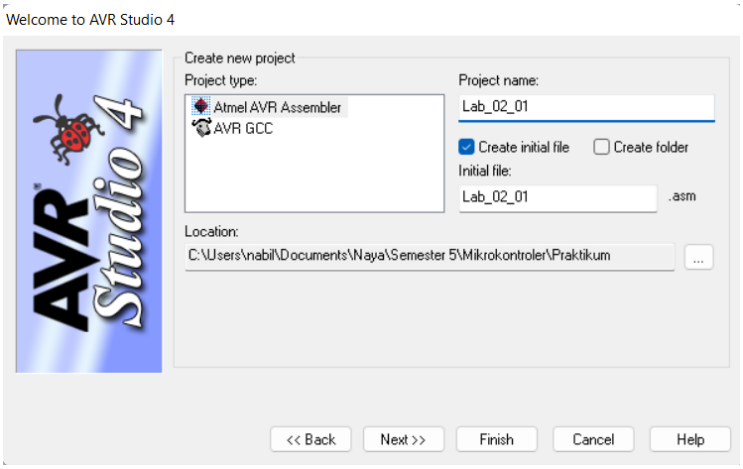
1. *Install* AVR Studio 4
2. Membuka aplikasi

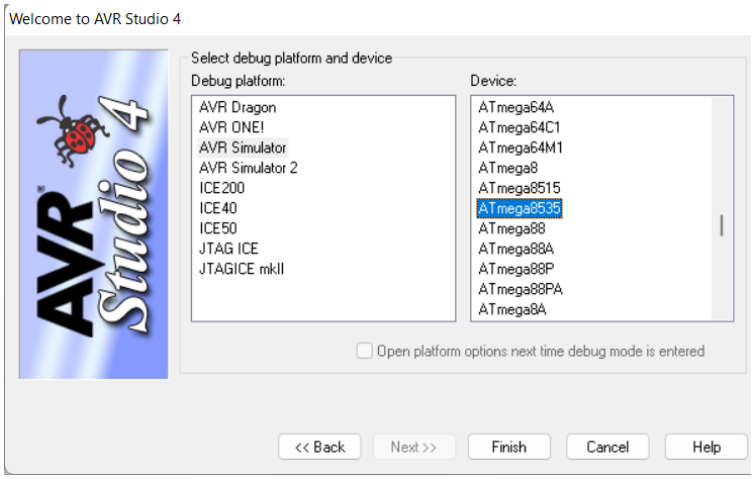


3. Membuat *project* baru

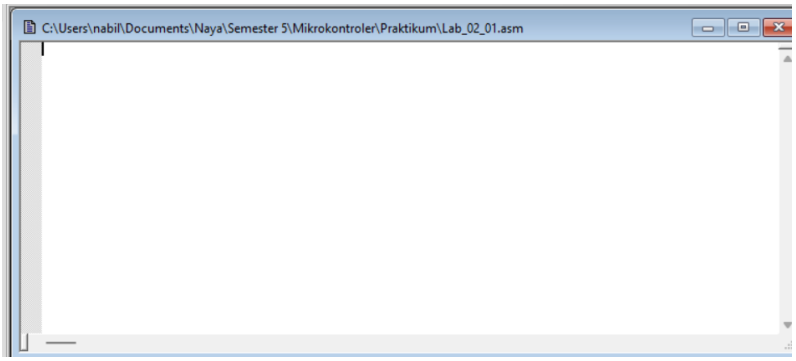


4. Memberikan nama pada project tersebut → klik **next** → klik **AVR simulator** → klik **ATmega8535** → klik **Finish**





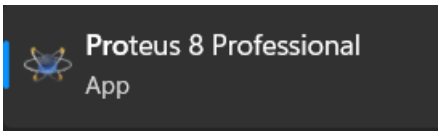
5. Menuliskan program pada *sheet* yang disediakan.



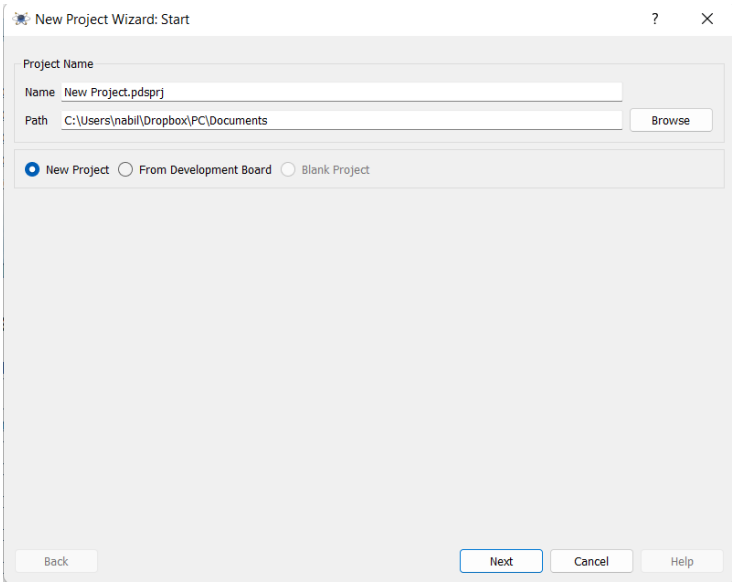
6. Kemudian, lakukan *running* program

 , F11

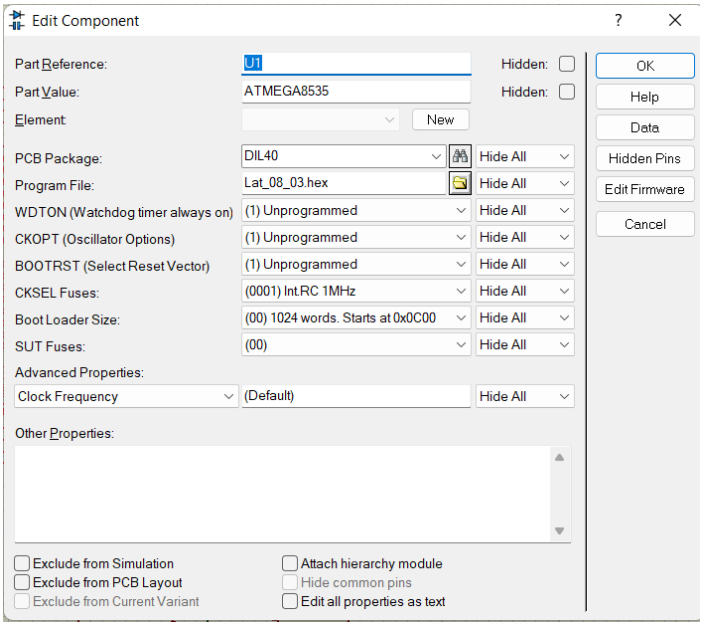
7. Membuka aplikasi proteus



8. Membuat *project* baru



9. Membuat rangkaian yang diinginkan
10. Klik 2 kali pada ATMega 8535 → klik **Program File** → pilih file yang format filenya .hex → klik **ok**



11. **Run** file proteus, dan perhatikan kesesuaian dengan program yang telah dibuat.

D. PEMBAHASAN DAN ANALISIS

1. Pembahasan

A. Membuat BCD UP Counter dan Down Counter 0 - 9

- Up Counter
- Kode Program

```
 .include "m8535def.inc"
.org 0x00
rjmp main
    ; PAKAI PROJECT "ROOT"
      ; DECIMAL UPCOUNTER, DENGAN 74LS47
      ; DELAY MENGGUNAKAN LOOP 3 REGISTER
     ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
   PROTEUS_PROJECT]

MAIN:       LDI R16,LOW(RAMEND)
        OUT SPL,R16
        LDI R16,HIGH(RAMEND)
        OUT SPH,R16

        LDI R20,0XFF
        OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:  LDI R18,0 ;buat counter
SANA:       OUT PORTB,R18 ;port b sebagai output
        RCALL DELAY
        INC R18 ;naikin
        CPI R18,0X0A ;
        BRNE SANA
        RJMP ITU

DELAY:      LDI R23,3 ;buat ngatur delay
```

```
LUP1:       LDI R24,0XFF ; buat output
LUP2:       LDI R25,0 ; masukin r25 biar jadi nol
LUP3:       DEC R25 ; naik
        BRNE LUP3
        DEC R24
        BRNE LUP2
        DEC R23
        BRNE LUP1
        RET
```

- Down Counter
- Kode Program

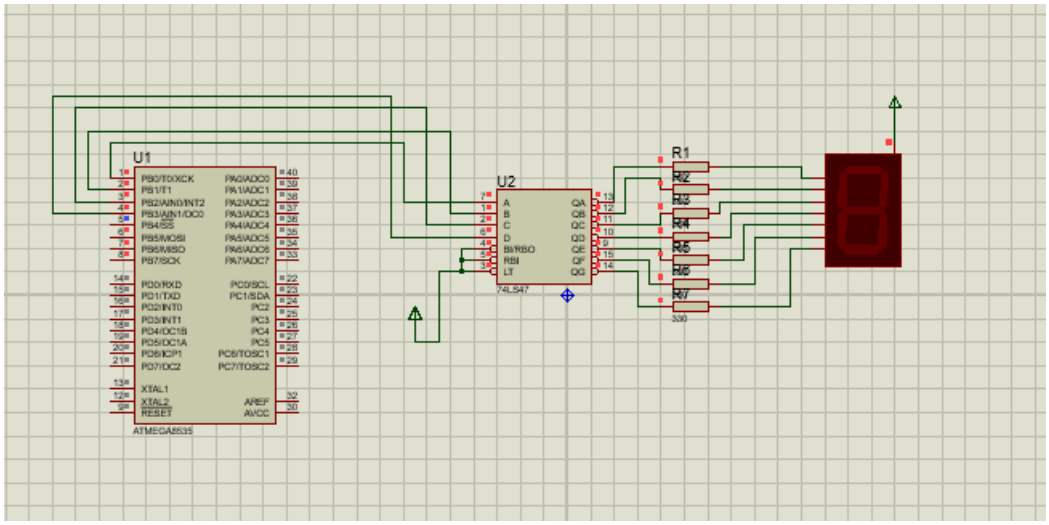
```
.include "m8535def.inc"
.org 0x00
rjmp main
    ; PAKAI PROJECT "ROOT"
       ; DECIMAL UPCOUNTER, DENGAN 74LS47
       ; DELAY MENGGUNAKAN LOOP 3 REGISTER
      ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
    PROTEUS_PROJECT]

MAIN:       LDI R16,LOW(RAMEND)
        OUT SPL,R16
        LDI R16,HIGH(RAMEND)
        OUT SPH,R16

        LDI R20,0XFF
        OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:  LDI R18,9 ;buat counter
SANA:       OUT PORTB,R18 ;port b sebagai output
```

```
        RCALL DELAY
        DEC R18 ;naikin
        CPI R18,0 ;
        BRNE SANA
        RJMP ITU

DELAY:  LDI R23,3 ;buat ngatur delay
LUP1:   LDI R24,0XFF ; buat output
LUP2:   LDI R25,0 ; masukin r25 biar jadi nol
LUP3:   DEC R25 ; naik
        BRNE LUP3
        DEC R24
        BRNE LUP2
        DEC R23
        BRNE LUP1
        RET
```



B. Membuat BCD UP Counter Modulo 26 (0-25). Gunakan 7-segemnet Common Anode

- Kode Program

```
.include "m8535def.inc"
.org 0x00
```

```
rjmp main
    ; PAKAI PROJECT "ROOT"
       ; DECIMAL UPCOUNTER, DENGAN 74LS47
       ; DELAY MENGGUNAKAN LOOP 3 REGISTER
      ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
    PROTEUS_PROJECT]

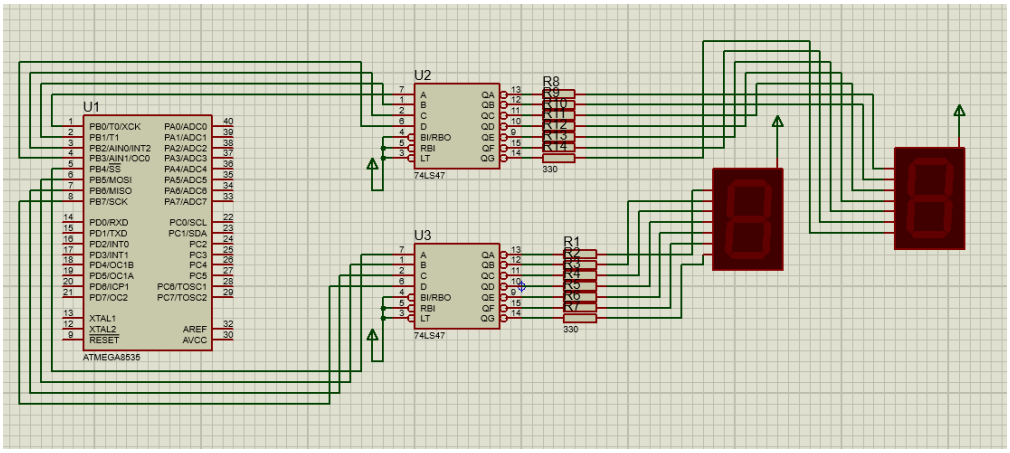
MAIN:       LDI R16,LOW(RAMEND)
        OUT SPL,R16
        LDI R16,HIGH(RAMEND)
        OUT SPH,R16

        LDI R20,0XFF
        OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:  LDI R18,0 ;buat counter
SANA:       OUT PORTB,R18 ;port b sebagai output
        RCALL DELAY
        LDI R20,1
        ADD R19,R20
        RCALL DAA
        MOV R18,R19
        CPI R18,26
        BRNE SANA
        RJMP ITU

DELAY:      LDI R23,3 ;buat ngatur delay
LUP1:       LDI R24,0XFF ; buat output
LUP2:       LDI R25,0 ; masukin r25 biar jadi nol
LUP3:       DEC R25 ; naik
        BRNE LUP3
        DEC R24
        BRNE LUP2
        DEC R23
```

```
        BRNE LUP1
        RET
daa:    push r16;amankan isi reg. ke stack
        push r17
        push r21
        push r24
        in r16,sreg; amankan sreg
        in r17,sreg
        ldi r24,0 ; r24 hanya sbg status dr carry flag
        adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
        out sreg,r17
        brhc lup4; lompat ke lup4 jika H=0
        rcall fk_ln; satuan + 06 jika H=1
    ;   rjmp lup5
lup4:   mov r21,r19
        andi r21,$0f; ambil dgt.satuan
        cpi r21,$0a ; > 9
        brcs lup5; lompat ke lup5 jika < 9
        rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
        brcc lup6; lompat ke lup6 jika C=0
        rcall fk_hn; puluhan + 60 jika C=1
    ;   rjmp lup7
lup6:   mov r21,r19
        andi r21,0xf0; ambil dgt.puluhan
        cpi r21,0xa0
        brcs lup7; lompat ke lup7 jika pul < a
        rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
        clc; C = 0
        in r16,sreg ;amankan sreg stlh c=0
        cpi r24,0 ;menguji status
```

```
        breq lup8
        out sreg,r16;ambil sreg, set c
        sec; C = 1
        in r16,sreg; amankan sreg stlh c=1
lup8:
        out sreg,r16;status akhir
        pop r24;kembalikan isi reg. dr stack
        pop r21
        pop r17
        pop r16
        ret
fk_ln: ldi r21,06; koreksi dgt.satuan
        out sreg,r16
        add r19,r21
        in r16,sreg
        brcc tuh
        ldi r21,0x01
        add r17,r21
tuh:    out sreg,r16
        adc r24,r24
        ret
fk_hn:          ldi r21,$60; koreksi dgt.puluhan
        out sreg,r16
        add r19,r21
        in r16,sreg
        adc r24,r24; amankan carry pd r24
        ret ; akhir dari prog. subrutin.
```



C. Membuat BCD Down Counter Modulo 31 (31-0). Gunakan 7-segemnet Common Anode

- Kode Program

```
 .INCLUDE"M8535DEF.INC"
.ORG 0
RJMP MAIN
        ; BCD DOWN COUNTER MODULO 30, DENGAN 74LS47
        ; DELAY MENGGUNAKAN LOOP 3 REGISTER
        ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
    PROTEUS_PROJECT]

MAIN:       LDI R16,LOW(RAMEND)
        OUT SPL,R16
        LDI R16,HIGH(RAMEND)
        OUT SPH,R16

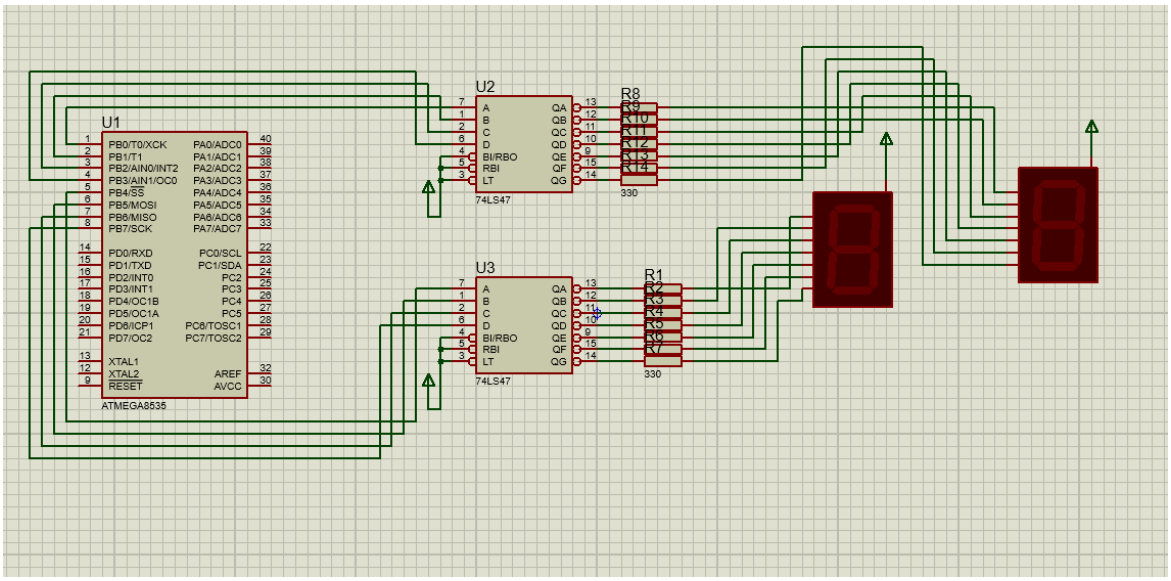
        LDI R20,0XFF
        OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:  LDI R18,0X30
SANA:       OUT PORTB,R18
        RCALL DELAY
        LDI R20,0X99
        ADD R18,R20
```

```
        MOV R19,R18
        RCALL DAA
        MOV R18,R19
        CPI R18,0X99
        BRNE SANA
        RJMP ITU

DELAY:  LDI R23,3
LUP1:   LDI R24,0XFF
LUP2:   LDI R25,0
LUP3:   DEC R25
        BRNE LUP3
        DEC R24
        BRNE LUP2
        DEC R23
        BRNE LUP1
        RET
daa:    push r16;amankan isi reg. ke stack
        push r17
        push r21
        push r24
        in r16,sreg; amankan sreg
        in r17,sreg
        ldi r24,0 ; r24 hanya sbg status dr carry flag
        adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
        out sreg,r17
        brhc lup4; lompat ke lup4 jika H=0
        rcall fk_ln; satuan + 06 jika H=1
    ;   rjmp lup5
lup4:   mov r21,r19
        andi r21,$0f; ambil dgt.satuan
        cpi r21,$0a ; > 9
```

```
        brcs lup5; lompat ke lup5 jika < 9
        rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
        brcc lup6; lompat ke lup6 jika C=0
        rcall fk_hn; puluhan + 60 jika C=1
    ;   rjmp lup7
lup6:   mov r21,r19
        andi r21,0xf0; ambil dgt.puluhan
        cpi r21,0xa0
        brcs lup7; lompat ke lup7 jika pul < a
        rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
        clc; C = 0
        in r16,sreg ;amankan sreg stlh c=0
        cpi r24,0 ;menguji status
        breq lup8
        out sreg,r16;ambil sreg, set c
        sec; C = 1
        in r16,sreg; amankan sreg stlh c=1
lup8:
        out sreg,r16;status akhir
        pop r24;kembalikan isi reg. dr stack
        pop r21
        pop r17
        pop r16
        ret
fk_ln:  ldi r21,06; koreksi dgt.satuan
        out sreg,r16
        add r19,r21
        in r16,sreg
        brcc tuh
        ldi r21,0x01
        add r17,r21
```

```
tuh:    out sreg,r16
        adc r24,r24
        ret
fk_hn:      ldi r21,$60; koreksi dgt.puluhan
        out sreg,r16
        add r19,r21
        in r16,sreg
        adc r24,r24; amankan carry pd r24
ret ; akhir dari prog. subrutin.
```



D. Membuat BCD Up/Down Counter Modulo 36 (0-35). Mode Up/down dikendalikan oleh push button.

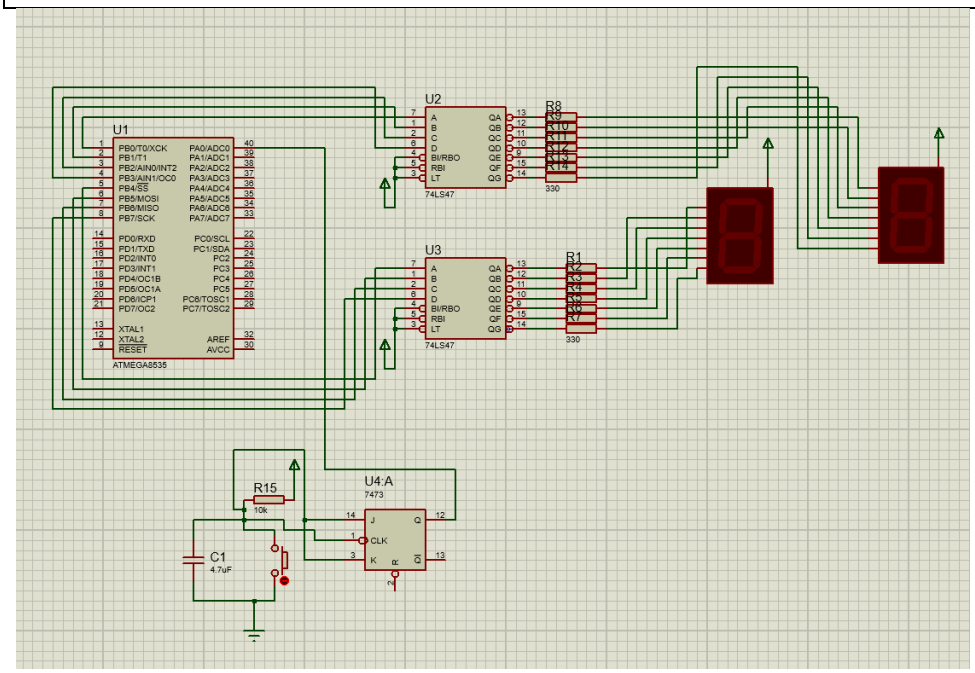
- Kode Program

```
 .INCLUDE"M8535DEF.INC" ;NUMBER THREE
.ORG 0
RJMP MAIN
    MAIN:   LDI R16,LOW(RAMEND)
        OUT SPL,R16
        LDI R16,HIGH(RAMEND)
        OUT SPH,R16
```

```
        LDI R20,0XFF
        OUT DDRB,R20 ; PORT B SBG OUTPUT
        LDI R20,0
        OUT DDRA,R20 ; PORT A SBG INPUT
        LDI R30,6
ITU:    LDI R18,0
SANA:       OUT PORTB,R18
        RCALL DELAY
        IN R17, PINA
        ANDI R17,1
        CPI R17,0
        BRNE SINI

        INC R18
        MOV R16,R18
        ANDI R16,0X0F
        CPI R16,0X0A
        BRNE SAN
        ADD R18,R30
SAN:    CPI R18,0X26
        BRNE SANA
        RJMP ITU
SINI:   DEC R18
        MOV R29,R18
        ANDI R29,0X0F
        CPI R29,0X0F
        BRNE INN
        SUB R18,R30
INN:    CPI R18,0
        BRLT UU
        RJMP SANA
UU:         LDI R18,0X35
        RJMP SANA
```

```
DELAY:    LDI R23,3
LUP1:     LDI R24,0XFF
LUP2:     LDI R25,0
LUP3:     DEC R25
          BRNE LUP3
          DEC R24
          BRNE LUP2
          DEC R23
          BRNE LUP1
RET
```



2. Analisis

Pada praktikum kali ini, terdapat beberapa program yang perlu dilakukan, namun sebelumnya ada beberapa perintah yang biasanya selalu ada agar dapar membuat suatu program berjalan. Praktikum kali ini menggunakan *software* simulasi mikrokontroler yaitu ATmega 8535 dan menggunakan konsep bahasa Assembly yang dimana bahasa pemrograman ini termasuk ke bahasa tingkat renda (*low level language*) atau bisa dibilang bahasa mesin. Simulator ATmega 8535 bukan simulator yang *sensitive case*

sehingga besar kecilnya huruf ketika penulisan program tidak akan berpengaruh kepada berjalannya suatu program (kecuali suatu nama subrutin, harus sama antara nama subrutin dengan subrutin yang dipanggil).

Dari setiap program yang digunakan, terdapat beberapa baris perintah "wajib" agar program dapat berjalan. Penjelasan dari tiap baris program yang digunakan tersebut sebagai berikut:

Perintah/Sintaks	**Deskripsi**
.include"m8535def.inc"	Mendefinisikan mikrokontroler yang direferensikan untuk program yaitu ATmega8535.
.org 0x0000	Mendefinisikan alamat awal program.
rjmp main	Melakukan *jump* (*relative jump*) ke label main *procedure* (prosedur utama).
ldi r16,low(ramend)	Menyiapkan *stack* (*stack pointer low* dan *stack pointer high*) untuk menerima data-data yang akan diletakkan/digunakan. Menempatkan 8bit pertama dan beberapa bit kedua ke R16, "low" berarti 8bit pertama di sebelah kanan dan "high" berarti beberapa bit kedua. "ramend" berarti alamat ram terakhir.
out spl,r16	
ldi r16,high(ramend)	
out sph,r16	
Akhir : rjmp Akhir	Untuk mengakhiri proses *running* programnya.

Pada program pertama membuat program BCD Up down counter 0-9, dengan membuat program wajib terlebih dahulu pada ATMega 8535. Kemudian mengisi register R20 dengan 0XFF tujuannya untuk DDRB agar membuat PORT B sebagai output. Kemudian mengisi suatu register (r18) dengan 0 yang nantinya digunakan untuk counter. Setelah itu, buat port B sebagai output, rcall delay untuk durasi waktu perubahan karakternya. Setelah itu, r18 di naikan, dan dicompare dengan 10, dan loop Kembali ke port b, apabila sudah Kembali loop ke counternya. Membuat subrutin untuk membuat register yang jadi outpurnya, kemudian jadi nol, setelah itu di naikan. Untuk proses down

counter tidak jauh berbeda, hanya berbeda untuk counternya diisi dengan 9 kemudian dibandingkannya dengan 0. Setelah itu, lakukan run program. Kemudian, membuat rangkaian pada proteus, dan lihat hasilnya sudah sesuai dengan yang diprogramkan.

Program kedua membuat BCD up Counter modulo 26 (0-25), dengan membuat program wajib terlebih dahulu pada ATMega 8535. Kemudian mengisi register r20 dengan 0XFF tujuannya untuk DDRB agar membuat PORT B menjadi output. Membuat suatu register (r18) diisi dengan 0 agar menjadi counter. Rcall delay untuk durasi waktu perubahan karakternya. Isi r20 dengan 1, kemudian dijumlahkan dengan r19, dan dikoreksi dengan factor koreksi pada subrutin daa. Setelah itu dipindahkan ke r18, dan di compare dengan 26 (karena modulonya 26). Setelah itu dibandingkan di looping seperti program 1.

Program ketiga menggunakan prinsip yang sama dengan program kedua hanya saja hingga modulo 31, sehingga pembandingnya 0X99.

E. KESIMPULAN

Pada praktikum yang telah dilakukan, diketahui bahwa simulator yang digunakan yaitu ATmega 8535, diketahui pula cara kerjanya, serta memahami konsep dari bahasa pemograman Assembly. Pada praktikum ini pula, diketahui cara dalam mengaplikasikan dari program ke *software.* Dalam melakukan pencacah naik bilangan desimal diawali dengan mendefinisikan nilai terkecil (0X00) pada sebuah register kemudian diberlakukan perintah **INC/DEC** sehingga nilai bergerak naik atau turun. Diketahui pula, subrutin untuk mengoreksi penjumlahan secara desimal.

DAFTAR PUSTAKA

Aji, Nianda Pratama. “PEREKAYASAAN SISTEM KONTROL .” *PEMOGRAMAN MIKROKONTROL ATMEGA 8535*, t.thn.

Andriyanto, Rendy. “SEJARAH MIKROPROSESOR DAN MIKROKONTROLER.” *RENDY ANDRIYANTO S-1 TEKNIK INFORMATIKA A 2014 SEJARAH MIKROPROSESOR DAN MIKROKONTROLER SEJARAH MIKROPROSESOR GENERASI 1*, 2014: 1-15.

Baeti, Ahmad Firdaus. *Konversi Sistem Bilangan: Desimal, Biner, Oktal dan Heksa Desimal.* 13 Januari 2022. http://www.miraudhatulwildan.sch.id/berita/detail/konversi-sistem-bilangan-desimal-biner-oktal-dan-heksa-desimal#:~:text=Cara%20mengkonversi%20bilangan%20desimal%20ke%20hexadesimal%20adalah%20dengan%20cara%20membagi,maka%20angkanya%20dirubah%20menjadi%20h (diakses September 06, 2022).

Widjojoatmodjo, Abdulkadir. *ATmega8535.* t.thn. https://p2k.unkris.ac.id/id1/1-3073-2962/Atmega8535_238516_p2k-unkris.html (diakses September 06, 2022).